

Oberon-0 语言的翻译模式设计

梁力航 23336128

2026 年 5 月 20 日

本文档对应 ROSE 项目实验四的提交要求：描述 Oberon-0 语言的翻译模式（Translation Scheme），包括文法转换、属性传递方案和递归子程序映射，并比较自顶向下与自底向上两种分析策略。

1 文法转换

为适应递归下降预测分析（自顶向下、LL），对原始 Oberon-0 EBNF 文法做了以下转换。

1.1 消除左递归

原始 grammar 中三处存在直接左递归，转换为带迭代循环的等价形式：

原始（左递归）	转换后（ * 迭代）
<code>simple_expr → term simple_expr + term simple_expr - term simple_expr OR term</code>	<code>simple_expr → term { + term - term OR term }*</code>
<code>term → factor term * factor term DIV factor term MOD factor term & factor</code>	<code>term → factor { * factor DIV factor MOD factor & factor }*</code>
<code>declarations → ... declarations proc_decl ";"</code>	<code>declarations → ... { proc_decl ";" }* (用 while 循环实现)</code>

表 1: 左递归消除规则

1.2 提取左公因子

Oberon-0 文法已天然避免了大部分左公因子问题。唯一的潜在歧义是 statement 规则中 IDENTIFIER 既可能开始一个赋值语句，也可能开始一个过程调用。通过一个 token 的向前看 (lookahead) 区分：

- IDENTIFIER selectors "!=" expr → 赋值语句
- IDENTIFIER selectors "(" params ")" → 过程调用
- IDENTIFIER selectors (后无 := 或 () → 无参过程调用

2 翻译模式设计

翻译模式的核心思路：每个非终结符对应一个递归子程序，继承属性用形式参数传递，综合属性用返回值表示。

2.1 属性传递方案

非终结符	继承属性	综合属性	语义动作
module	—	Module 对象	创建 Module，依次添加 Procedure
procedure_decl	Module	—	创建 Procedure，注册到 Module
statement_seq	Proc./StmtSeq	—	按序解析 statements
if_statement	—	IfStatement 对象	创建 IfStmt，递归填充分支
while_statement	—	WhileStmt 对象	创建 WhileStmt，递归填充循环体
expression	—	String (文本)	重建表达式文本用于流程图显示

表 2: 核心非终结符的属性传递方案

2.2 递归子程序映射

文法非终结符	递归子程序方法
module	parseModule()
declarations	parseDeclarations()
procedure_declaration	parseProcedureDeclaration()
statement_sequence	parseStatementSequence(Procedure)
statement	parseStatementCommon(Object container)
if_statement	parseIfStatement(Procedure)
while_statement	parseWhileStatement(Procedure)
expression	parseExpressionText() → String
simple_expression	while 循环调用 parseTermText()
term	while 循环调用 parseFactorText()
factor	parseFactorText() → String
type	parseType() → String

表 3: 完整递归子程序映射 (12 个方法)

2.3 流程图生成嵌入

语义动作在解析过程中直接调用 ROSE Flowchart API:

- 赋值/过程调用 → add(new PrimitiveStatement(text))
- IF 语句 → IfStatement(condition) → 递归填充 getTrueBody() / getFalseBody()
- WHILE 语句 → WhileStatement(condition) → 递归填充 getLoopBody()
- ELSIF 链 → 嵌套 IfStatement 到前一个 IF 的 false 分支

3 自顶向下 vs 自底向上——分析策略比较

通过实验三 (JavaCUP / LALR) 和实验四 (递归下降) 的亲身实践, 对如下维度的体会:

维度	递归下降（本实验）	LALR（实验三）
分析方向	自顶向下	自底向上
简单性	直观，方法直接映射非终结符，调试时打断点即可跟踪	需理解分析表、状态机和移进/归约逻辑
通用性	需手工消除左递归，不能处理所有 CFG	可处理 LR(1) 文法，更通用
语义动作	可在解析过程中任意位置嵌入	仅在产生式规约时执行
错误恢复	较易实现，方法级 panic-mode	需特殊 error 产生式或分析表条目
分析速度	$O(n)$ ，无表查找开销	$O(n)$ ，有状态表查找开销
表格大小	无需分析表	LR 分析表可能较大

表 4: 六维对比

具体体会：

1. 流程图生成的语义动作在递归下降中更自然——解析到 IF 关键字后可立即创建 IfStatement 对象，然后递归解析 true-branch 时直接向其中添加语句
2. 左递归消除是递归下降的额外成本——但 Oberon-0 仅需处理表达式和声明列表 3 处，成本可控
3. 错误恢复在递归下降中更灵活——可直接跳过当前输入直到同步符号（END、分号），不需额外设计
4. **LALR** 的表驱动方式在文法演进时更鲁棒——新增语法规则不一定需要修改现有规则的处理逻辑